D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI

Department of Computer Engineering

Lab Manual

Third Year Semester-VI

Subject: Machine Learning Lab

EVEN Semester

Institutional Vision, Mission and Quality Policy

Our Vision

To foster and permeate higher and quality education with value added engineering, technology programs, providing all facilities in terms of technology and platforms for all round development with societal awareness and nurture the youth with international competencies and exemplary level of employability even under highly competitive environment so that they are innovative adaptable and capable of handling problems faced by our country and world at large.

Our Mission

The Institution is committed to mobilize the resources and equip itself with men and materials of excellence thereby ensuring that the Institution becomes pivotal center of service to Industry, academia, and society with the latest technology. RAIT engages different platforms such as technology enhancing Student Technical Societies, Cultural platforms, Sports excellence centers, Entrepreneurial Development Center and Societal Interaction Cell. To develop the college to become an autonomous Institution & deemed university at the earliest with facilities for advanced research and development programs on par with international standards. To invite international and reputed national Institutions and Universities to collaborate with our institution on the issues of common interest of teaching and learning sophistication.

Our Quality Policy

ज्ञानधीनं जगत् सर्वम।

Knowledge is supreme.

Our Quality Policy

It is our earnest endeavour to produce high quality engineering professionals who are innovative and inspiring, thought and action leaders, competent to solve problems faced by society, nation and world at large by striving towards very high standards in learning, teaching and training methodology.

Our Motto: If it is not of quality, it is NOTRAIT!

Departmental Vision, Mission

Vision

To impart higher and quality education in computer science with value added engineering and technology programs to prepare technically sound, ethically strong engineers with social awareness. To extend the facilities, to meet the fast changing requirements and nurture the youths with international competencies and exemplary level of employability and research under highly competitive environments.

Mission

- To mobilize the resources and equip the institution with men and materials of excellence to provide knowledge and develop technologies in the thrust areas of computer science and Engineering.
- To provide the diverse platforms of sports, technical, co-curricular and extracurricular activities for the overall development of student with ethical attitude.
- To prepare the students to sustain the impact of computer education for social needs encompassing industry, educational institutions and public service.
- To collaborate with IITs, reputed universities and industries for the technical and overall upliftment of students for continuing learning and entrepreneurship.

Departmental Program Educational Objectives (PEOs)

1. **Learn and Integrate**
   To provide Computer Engineering students with a strong foundation in the mathematical, scientific and engineering fundamentals necessary to formulate, solve and analyze engineering problems and to prepare them for graduate studies.

2. **Think and Create**
   To develop an ability to analyze the requirements of the software and hardware, understand the technical specifications, create a model, design, implement and verify a computing system to meet specified requirements while considering real-world constraints to solve real world problems.

3. **Broad Base**
   To provide broad education necessary to understand the science of computer engineering and the impact of it in a global and social context.

4. **Techno-leader**
   To provide exposure to emerging cutting edge technologies, adequate training & opportunities to work as teams on multidisciplinary projects with effective communication skills and leadership qualities.

5. **Practice citizenship**
   To provide knowledge of professional and ethical responsibility and to contribute to society through active engagement with professional societies, schools, civic organizations or other community activities.

6. **Clarify Purpose and Perspective**
   To provide strong in-depth education through electives and to promote student awareness on thelife-long learning to adapt to innovation and change, and to be successful in their professional work or graduate studies.

Departmental Program Outcomes (POs)

PO1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences..

PO3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSOs)

PSO1: To build competencies towards problem solving with an ability to understand, identify, analyze and design the problem, implement and validate the solution including both hardware and software.

PSO2: To build appreciation and knowledge acquiring of current computer techniques with an ability to use skills and tools necessary for computing practice.

PSO3: To be able to match the industry requirements in the area of computer science and engineering. To equip skills to adopt and imbibe new technologies.

Index

List of Experiments

Sr. No.	Experiments Name
1	Perform data pre-processing on a dataset
2	Implementation of Principal Component Analysis.
3	a) To implement Linear Regression. b) b) To implement Logistic Regression.
4	Implement a basic McCulloch-Pitt's Model for any logical function.
5	Implement Naive Bayes classification algorithm.
6	To Implement Support Vector Machine for binary classification.(Competitive)
7	Implement K-means clustering algorithm.(Competitive)
8	Implementing Agglomerative Clustering.
9	Case study of reinforcement learning Elevator Dispatching.
10	**VLAB-RANDOM FOREST/BACK PROPAGATION**
11	**Content Beyond Syllabus - Optical Character Recognition**

Course Outcome & Experiment Plan

Course Outcomes:

CO1	Understand and implement data preprocessing techniques.
CO2	Apply different regression techniques for prediction.
CO3	Use neural network techniques for learning.
CO4	Apply classification algorithms and analyse the performance.
CO5	Identify pattern similarity and form clusters using clustering techniques.
CO6	Understand working of reinforcement learning.

Experiment Plan:

Module No.	Week No.	Experiments Name	Course Outcome
1	W1,W2	Perform data preprocessing on a dataset**.**	CO1
2	W3	Implement Principal Component Analysis.	CO2
3	W4	a) Implement Linear Regression algorithm for prediction	CO3
	W5	b) Implement Logistic Regression algorithm for prediction	CO3
4	W6	Implement McCulloch-Pitts Model.	CO3
5	W7	Implement Navie Bayes Classification algorithm.	CO4
6	W8	Implement Support Vector Machine for binary Classification.	CO4
7	W9	Implement K-means clustering algorithm.	CO5
8	W10	Implement Agglomerative Clustering.	CO5
9	W11	Case study of reinforcement learning Elevator Dispatching.	CO6
10	W11	VLAB – **RANDOM FOREST/BACK PROPAGATION**	CO6
11	W12	Content beyond Syllabus - **Optical Character Recognition**	

Mapping of Course outcomes with Program outcomes:

Subject Weight	Course Outcomes		Contribution to Program outcomes (PO)											
			1	2	3	4	5	6	7	8	9	10	11	12
PRATICAL 40%	CO1	Understand and implement data preprocessing techniques.	1	1	2	2	2						1	1
	CO2	Apply different regression techniques for prediction.	1	1	2	2	2						1	1
	CO3	Use neural network techniques for learning.	1	1	1	3	2						1	1
	CO4	Apply classification algorithms and analyse the performance.	1	2	2	1	2						1	1
	CO5	Identify pattern similarity and form clusters using clustering techniques.	1	2	2	1	2						1	1
	CO6	Understand working of reinforcement learning.	1	2	2	1	2						1	1

Mapping of Course outcomes with Program Specific outcomes:

Course Outcomes		Contribution to Program Specific outcomes (PSO)		
		1	2	3
CO1	Understand and implement data preprocessing techniques.	2	2	3
CO2	Apply different regression techniques for prediction.	2	2	3
CO3	Use neural network techniques for learning.	2	2	3
CO4	Apply classification algorithms and analyse the performance.	3	2	3
CO5	Identify pattern similarity and form clusters using clustering techniques.	2	2	3
CO6	Understand working of reinforcement learning.	2	2	3

Study and Evaluation Scheme

Course Code	Course Name	Teaching Scheme			Credits Assigned			
		Theory	Practical	Tutorial	Theory	Practical	Tutorial	Total
CEL601	Machine Learning Lab	03	02	--	03	01	--	04

Course Code	Course Name	Examination Scheme		
		Term Work	Oral & Practical	Total
CEL601	Machine Learning Lab	25	25	50

Term Work:

1. The Term work Marks are based on the weekly experimental performance of the students, Oral performance and regularity in the lab.
2. Students are expected to be prepared for the lab ahead of time by referring the manual and perform the experiment under the guidance and discussion. Next week the experiment write-up to be corrected along with oral examination.

Practical & Oral:

1. Practical and Oral exam will be based on the entire syllabus of Computational Intelligence.

Machine Learning
Experiment No.: 1
Perform Data Pre-processing on a Dataset

Experiment No. 1

1. **Aim:** Perform data pre-processing on a dataset

2. **Objectives:** From this experiment, the student will be able to
   - Showcase how data processing is done when different techniques are applied to solve real-world problems in diverse domains such as healthcare, finance, robotics, and environmental science in Machine Learning.

3. **Outcomes:** The learner will be able to

   - Improve comprehension of how Machine learning algorithms work and their applications in solving real-world problems. Applying fundamental engineering concepts appropriate to the discipline.
   - Evaluate strengths and weaknesses of various algorithms, helping researchers and practitioners choose the most suitable methods for specific applications.

4. **Software Required:** Google Colab/Python/R

5. **Theory :**

   Data pre-processing is a process of preparing the raw data and making it suitable for a machine learning model. It is the first and crucial step while creating a machine learning model. When creating a machine learning project, it is not always a case that we come across the clean and formatted data. And while doing any operation with data, it is mandatory to clean it and put in a formatted way. So for this, we use data pre-processing task.

   A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models. Data pre-processing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model. It involves below steps:

   - Getting the dataset
   - Importing libraries
   - Importing datasets
   - Finding Missing Data
   - Splitting dataset into training and test set
   - Feature scaling

Get the Dataset

The collected data for a particular problem in a proper format is known as the **dataset**.

Importing Libraries

In order to perform data preprocessing using Python, we need to import some predefined Python libraries.

Importing the Datasets

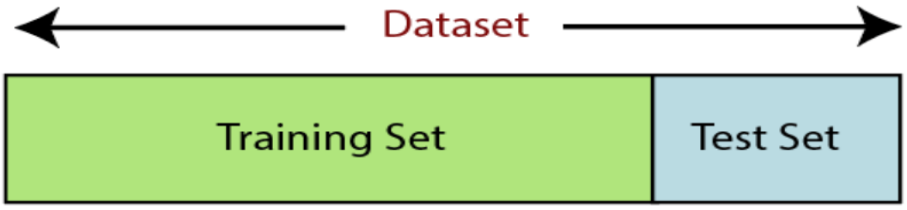
Save the dataset with .csv format and then import the data with read_csv() function of pandas library.

Handling Missing data: Calculating the mean we will calculate the mean of that column or row which contains any missing value and will put it on the place of missing value. This strategy is useful for the features which have numeric data such as age, salary, year, etc. Here, we will use this approach.

Splitting the Dataset into the Training set and Test set

In machine learning data preprocessing, we divide our dataset into a training set and test set. This is one of the crucial steps of data preprocessing as by doing this, we can enhance the performance of our machine learning model.Suppose, if we have given training to our machine learning model by a dataset and we test it by a completely different dataset. Then, it will create difficulties for our model to understand the correlations between the models.

If we train our model very well and its training accuracy is also very high, but we provide a new dataset to it, then it will decrease the performance. So we always try to make a machine learning model which performs well with the training set and also with the test dataset. Here, we can define these datasets as:



Training Set: A subset of dataset to train the machine learning model, and we already know the output.

Test set: A subset of dataset to test the machine learning model, and by using the test set, model predicts the output.

Feature Scaling: Feature scaling is the final step of data preprocessing in machine learning. It is a technique to standardize the independent variables of the dataset in a specific range. In feature scaling, we put our variables in the same range and in the same scale so that no any variable dominate the other variable.

we will create the object of **StandardScaler** class for independent variables or features. And then we will fit and transform the training dataset.

6.

7. Conclusion:

Students are able to understand the need of pre-processing steps and implemented same with python on the available dataset.

8. Viva Questions:

1. What is Data Preprocessing?
2. What do you mean by feature scaling or data normalization?
3. Explain some techniques for feature scaling?
4. What are the missing values? and How do you handle missing values?

9. Additional Learning:

10. References:

Books

1. **Russell, S., & Norvig, P.** (2016). *Artificial Intelligence: A Modern Approach*. Pearson.
2. **Mitchell, T. M.** (1997). *Machine Learning*. McGraw-Hill.

- **Reference link**
- **https://www.youtube.com/watch?v=NSxEiohAH5o**
- **https://towardsdatascience.com/data-preprocessing-in-python-b52b652e37d5**

Machine Learning
Experiment No.: 2
Implementation of Principal Component Analysis

Experiment No. 2

1. **Aim:** Implementation of Principal Component Analysis

2. **Objectives:**
   - To understand the basic concepts and importance of pre-processing of data analysis
   - To apply simple PCA reduction technique on actual dataset

3. **Outcomes**: Students will be able to implement PCA reduction techniques.

4. **Hardware / Software Required:** Python 3.10

5. **Theory:**

Principal Component Analysis is an unsupervised learning algorithm that is used for the dimensionality reduction in machine learning. It is a statistical process that converts the observations of correlated features into a set of linearly uncorrelated features with the help of orthogonal transformation. These new transformed features are called the Principal Components. It is one of the popular tools that is used for exploratory data analysis and predictive modelling. It is a technique to draw strong patterns from the given dataset by reducing the variances.

Some real-world applications of PCA are image processing, movie recommendation system, optimizing the power allocation in various communication channels. It is a feature extraction technique, so it contains the important variables and drops the least important variable.

The PCA algorithm is based on some mathematical concepts such as:

- Variance and Covariance
- Eigenvalues and Eigen factors

Some common terms used in PCA algorithm:

Dimensionality: It is the number of features or variables present in the given dataset. More easily, it is the number of columns present in the dataset.

Correlation: It signifies that how strongly two variables are related to each other. Such as if one changes, the other variable also gets changed. The correlation value ranges from -1 to +1. Here, -1 occurs if variables are inversely proportional to each other, and +1 indicates that variables are directly proportional to each other.

Orthogonal: It defines that variables are not correlated to each other, and hence the correlation between the pair of variables is zero.

Eigenvectors: If there is a square matrix M, and a non-zero vector v is given. Then v will be eigenvector if Av is the scalar multiple of v.

Covariance Matrix: A matrix containing the covariance between the pair of variables is called the Covariance Matrix.

The transformed new features or the output of PCA are the Principal Components. The number of these PCs are either equal to or less than the original features present in the dataset. Some properties of these principal components are given below:

The principal component must be the linear combination of the original features.

These components are orthogonal, i.e., the correlation between a pair of variables is zero.

The importance of each component decreases when going to 1 to n, it means the 1 PC has the most importance, and n PC will have the least importance.

Applications of Principal Component Analysis:

- PCA is mainly used as the dimensionality reduction technique in various AI applications such as computer vision, image compression, etc.
- It can also be used for finding hidden patterns if data has high dimensions. Some fields where PCA is used are Finance, data mining, Psychology, etc.

6. Steps to implement PCA in python

I. Import the python libraries

II Import dataset

III. Apply PCA

- Standardize the dataset prior to PCA.
- Import PCA from sklearn. Decomposition.
- Choose the number of principal components.

IV. Check Components

The principal components_ provide an array in which the number of rows tells the number of principal components while the number of columns is equal to the number of features in actual data. We can easily see that there are three rows as n components was chosen to be 3. However, each row has 30 columns as in actual data.

V. Plot the components (Visualization)

Plot the principal components for better data visualization.

For three principal components, we need to plot a 3d graph. x[:,0] signifies the first principal component. Similarly, x[:,1] and x[:,2] represent the second and the third principal component.

VI. Calculate variance ratio

7. Conclusion: Implemented PCA and understand the use and its applications.

8. Viva Questions:

1. What is a PCA?
2. Can you list out the critical assumptions of linear regression?
3. What is the primary difference between R square and adjusted R square?
4. How Principal Component Analysis (PCA) is used for Dimensionality Reduction?

- **Reference link**
  - https://www.geeksforgeeks.org/principal-component-analysis-with-python/
  - https://www.geeksforgeeks.org/principal-component-analysis-with-python/
  - https://builtin.com/machine-learning/pca-in-python

Machine Learning
Experiment No.: 3 (a)
To Implement Linear Regression.

Experiment 3 (a)

1. **Aim:** To implement Linear Regression.

2. **Software Required:** (Students should write **Software required** based on software used for implementing program) **e.g.,** python

3. **Theory:**

Linear regression is a basic predictive analytics technique that uses historical data to predict an output variable. It is popular for predictive modeling because it is easily understood. Linear regression models have many real-world applications in an array of industries such as economics (e.g., predicting growth), business (e.g., predicting product sales, employee performance), social science (e.g., predicting political leanings from gender or race), healthcare (e.g., predicting blood pressure levels from weight, disease onset from biological factors), and more.

The basic idea is that if we can fit a linear regression model to observed data, we can then use the model to predict any future values. There are two kinds of variables in a linear regression model:

- The input or predictor variable is the variable(s) that help predict the value of the output variable. It is commonly referred to as X.
- The output variable is the variable that we want to predict. It is commonly referred to as Y.

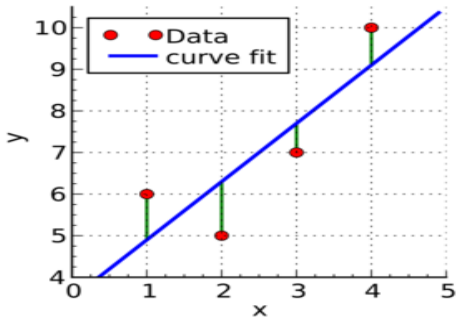
To estimate Y using linear regression, we assume the equation:

$Y_e = \alpha + \beta X$

where $\mathbf{Y_e}$ is the estimated or predicted value of **Y** based on our linear equation.

Our goal is to find statistically significant values of the parameters α and β that minimize the difference between Y and Y_e. If we are able to determine the optimum values of these two parameters, then we will have the line of best fit that we can use to predict the values of Y, given the value of X. How do we estimate α and β? We can use a method called ordinary least squares.

Ordinary Least Squares



Green lines show the difference between actual values **Y** and estimate values $\mathbf{Y_e}$

The objective of the least squares method is to find values of α and β that minimize the sum of the squared difference between Y and Y_e. We will not go through the derivation here, but using calculus we can show that the values of the unknown parameters are as follows:

$$\beta = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^{n}(X_i - \bar{X})^2}$$

$$\alpha = \bar{Y} - \beta * \bar{X}$$

where $\bar{X}$ is the mean of X values and $\bar{Y}$ is the mean of Y values.

If you are familiar with statistics, you may recognize β as simply Cov(X, Y) / Var(X).

Implementation Steps:

1. Generate data with nonzero mean and standard deviation (X). Also, create random data by multiplying a constant and adding residual which is an actual data (Y).
2. Calculate the mean of X and Y

   Mean_X = X1+X2+…X_N/ N

   Mean_Y = Y1+Y2+…Y_N/ N
3. Calculate α and β *using*

$$\beta = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^{n}(X_i - \bar{X})^2}$$

$$\alpha = \bar{Y} - \beta * \bar{X}$$

4. Compute the predicted output y

$$Y_e = \alpha + \beta X$$

5. Plot regression against actual data

4. **Output Analysis:** (Students should write output analysis based on the output. Specify each output explicitly with output analysis)

5. **Additional Learning:** (Students should write additional learning on their own based on what additionally they learnt after performing the experiment)

- Regression analysis allows you to understand the strength of relationships between variables.
- Regression analysis tells you what predictors in a model are statistically significant and which are not.
- Regression analysis can give a confidence interval for each regression coefficient that it estimates
- The simple linear regression method tries to find the relationship between a single independent variable and a corresponding dependent variable. The independent variable is the input, and the corresponding dependent variable is the output.
- The multiple linear regression method tries to find the relationship between two or more independent variables and the corresponding dependent variable. There's also a special case of multiple linear regression called polynomial regression. There are other non-linear regression methods used for highly complicated data analysis.

6. **Conclusion:**(Students should write proper conclusion (in the form of discussion) on their own)

- Linear regression is a statistical method that tries to show a relationship between variables. It looks at different data points and plots a trend line. A simple example of linear regression is finding that the cost of repairing a piece of machinery increases with time.
- More precisely, linear regression is used to determine the character and strength of the association between a dependent variable and a series of other independent variables. It helps create models to make predictions.

Machine Learning

Experiment No.: 3 (b)

To implement Logistic Regression.

Experiment 3 (b)

1. Aim: Perform pre-processing, design, and implement a logistic regression model for solar radiation prediction.

2. Objectives: From this experiment, the student will be able to

- Understand the preprocessing of data for logistic regression.
- Understand use of logistic regression by implementing a model for real-time datasets.

3. Outcomes: The learner will be able to

- Understand, identify, analyze, and design the problem, implement and validate the solution for regression
- Applying fundamental engineering concepts appropriate to the discipline.
- Potential to formulate and solve engineering problems.

4. Software Required: Google Colab/Python/R

5. Theory: In statistics, logistic regression is used to model the probability of a certain class or event. Logistic regression is similar to linear regression because both of these involve estimating the values of parameters used in the prediction equation based on the given training data. Linear regression predicts the value of some continuous, dependent variable. Whereas logistic regression predicts the probability of an event or class that is dependent on other factors. Thus, the output of logistic regression always lies between 0 and 1. Because of this property, it is commonly used for classification purpose.

Logistic Model

Consider a model with features *x1, x2, x3 … xn*. Let the binary output be denoted by *Y*, that can take the values 0 or 1. Let *p* be the probability of *Y = 1*, we can denote it as *p = P(Y=1)*. The mathematical relationship between these variables can be denoted as:

$$ln(\frac{p}{1-p}) = b_0 + b_1 x_1 + b_2 x_2 + b_3 x_3 \ldots b_n x_n$$

Here the term *p/(1−p)* is known as the *odds* and denotes the likelihood of the event taking place. Thus *ln(p/(1−p))* is known as the *log odds* and is simply used to map the probability that lies between 0 and 1 to a range between (−∞,+∞). The terms *b0, b1, b2…* are parameters (or weights) that we will estimate during training.

So we simplify the equation to obtain the value of p:

1. The log term ln on the LHS can be removed by raising the RHS as a power of e:

$$\frac{p}{1-p} = e^{b_0+b_1x_1+b_2x_2+b_3x_3...b_nx_n}$$

2. Now we can easily simplify to obtain the value of p :

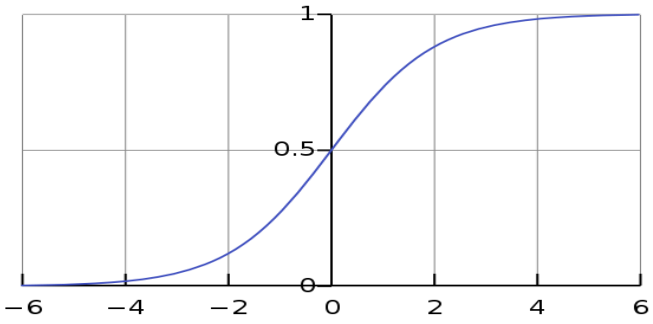
$$p = \frac{e^{b_0+b_1x_1+b_2x_2+b_3x_3...b_nx_n}}{1+e^{b_0+b_1x_1+b_2x_2+b_3x_3...b_nx_n}}$$

$$or$$

$$p = \frac{1}{1+e^{-(b_0+b_1x_1+b_2x_2+b_3x_3...b_nx_n)}}$$

3. This actually turns out to be the equation of the Sigmoid Function which is widely used in other machine learning applications. The Sigmoid Function is given by:

$$S(x) = \frac{1}{1+e^{-x}}$$



4. Now we will be using the above derived equation to make our predictions. Before that we will train our model to obtain the values of our parameters b0, b1, b2… that result in least error. This is where the error or loss function comes in.

Loss Function

The loss is basically the error in our predicted value. In other words, it is a difference between our predicted value and the actual value. We will be using the L2 Loss Function to calculate the error. Theoretically you can use any function to calculate the error. This function can be broken down as:

1. Let the actual value be y_i. Let the value predicted using our model be denoted as $\bar{y}_i$. Find the difference between the actual and predicted value.
2. Square this difference.
3. Find the sum across all the values in training data.

$$L = \sum_{i=1}^{n} (y_i - \bar{y_i})^2$$

Now that we have the error, we need to update the values of our parameters to minimize this error. This is where the "learning" actually happens, since our model is updating itself based on its previous output to obtain a more accurate output in the next step. Hence with each iteration our model becomes more and more accurate. We will be using the ***Gradient Descent Algorithm*** to estimate our parameters. Another commonly used algorithm is the Maximum Likelihood Estimation.

The Gradient Descent Algorithm

You might know that the partial derivative of a function at its minimum value is equal to 0. So gradient descent basically uses this concept to estimate the parameters or weights of our model by minimizing the loss function. For simplicity, assume that our output depends only on a single feature *x*. So, we can rewrite our equation as:

$$\bar{y_i} = p = \frac{1}{1 + e^{-(b_0 + b_1 x_i)}} = \frac{1}{1 + e^{-b_0 - b_1 x_i}} \quad (1)$$

Thus, we need to estimate the values of weights b0 and b1 using our given training data.

1. Initially let b0=0 and b1=0. Let L be the learning rate. The learning rate controls by how much the values of b0 and b1 are updated at each step in the learning process. Here let L=0.001.
2. Calculate the partial derivative with respect to b0 and b1. The value of the partial derivative will tell us how far the loss function is from its minimum value. It is a measure of how much our weights need to be updated to attain minimum or ideally 0 error. In case you have more than one feature, you need to calculate the partial derivative for each weight b0, b1 … bn where n is the number of features.

$$D_{b_0} = -2 \sum_{i=1}^{n} (y_i - \bar{y_i}) \times \bar{y_i} \times (1 - \bar{y_i})$$
$$D_{b_1} = -2 \sum_{i=1}^{n} (y_i - \bar{y_i}) \times \bar{y_i} \times (1 - \bar{y_i}) \times x_i \quad (2)$$

3. Next we update the values of b0 and b1:

$$b_0 = b_0 - L \times D_{b_0}$$
$$b_1 = b_1 - L \times D_{b_1} \quad (3)$$

4. We repeat this process until our loss function is a very small value or ideally reaches 0 (meaning no errors and 100% accuracy). The number of times we repeat this learning process is known as **iterations or epochs.**

Steps:

1. Read and visualize the database
2. Divide the data to training set and test set (80:20)
3. Perform data normalization
4. Compute the method to make predictions using equation (1)

5. Evaluate the method to train the model using equations (2) and (3). Obtain b0 and b1
6. Train the model using training data
7. Make the predictions
8. Calculate accuracy

6. Output Analysis: (Students should write output analysis based on the output. Specify each output explicitly with output analysis)

7. **Additional Learning:** (Students should write additional learning on their own based on what additionally)
   - Linear regression predicts the continuous dependent variable for a given set of independent variables, logistic regression predicts the categorical dependent variable.
   - Linear regression is used to solve regression problems, logistic regression is used to solve classification problems.

8. **Conclusion:** (Students should write proper conclusion (in the form of discussion) on their own)

   - Logistic regression can solve regression problems, but it's mainly used for classification problems. Its output can only be 0 or 1. It's valuable in situations where we need to determine the probabilities between two classes or, in other words, calculate the likelihood of an event.

9. Viva Questions:
   - What is Logistic Regression?
   - What are different applications of Logistic Regression?

10. References:

1. Jake Vander Plas, Python Data Science Handbook: Essential Tools for Working with Data, O'reilly Publication
2. Wes McKinney , Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython, O'reilly Publication
3. Jason Test, PYTHON FOR DATASCIENCE
4. Jason Test, Python Programming: 3 BOOKS IN1

Machine Learning

Experiment No.: 4

McCulloch-Pitt's Model

Experiment 4

1. **Aim:** Implement a basic McCulloch-Pitt's Model for any logical function.
2. **Objectives:**
   - Understand the fundamentals of Artificial Neural Network.
   - Explore various machine learning tools and techniques.

3. Outcomes:

- Students will be able to use neural network techniques for learning.

4. Software (tools/packages) required : Python

5. Theory:

Basic Theory of NN-

Neural networks are parallel computing devices, which is basically an attempt to make a computer model of the brain. The main objective is to develop a system to perform various computational tasks faster than the traditional systems. These tasks include pattern recognition and classification, approximation, optimization, and data clustering.

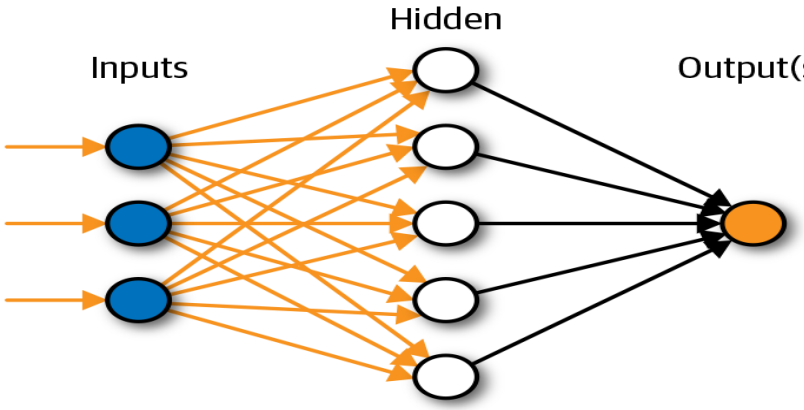
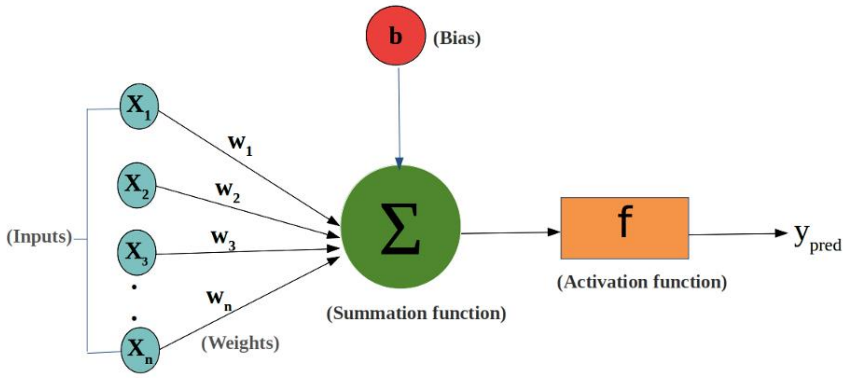


Fig: Basic structure of Neural Net

Basic operation of NN-

χ X1 and X2 – input neurons.
χ Y- output neuron
χ Weighted interconnection links- W1 and W2.
χ Net input calculation is : y1 = -x1w1 + x2w2
χ Output is : y= f(Y_{in})
The function to be applied over the net input is called activation function.

McCulloch-Pitt's Model-



Step1: Provide Training data (Truth table of any logical operation)
Step2: Assume weights
Step3: Draw NN architecture
Step4: Calculate net input

$Y_{in} = X_1W_1 + X_2W_2 + \ldots\ldots\ldots + X_nW_n$

Step5: Choose threshold to apply activation function
Step6: Calculate output using activation function

$F(y_{in}) = \{ 1, \text{if } Y_{in} >= \theta$

$0, \text{if } Y_{in} < \theta \}$

5. Output Analysis: (Students should write output analysis based on the output. Specify each output explicitly with output analysis)

6. Additional Learning: (Students should write additional learning on their own based on what additionally they learnt after performing the experiment)

7. Conclusion: (Students should write proper conclusion (in the form of discussion)on their own)

Most of the work is carried on the basis of MCP model for observing the nature of logic gates like OR, AND, NOT, NAND, NOR, XOR with variable threshold conditions and for variable weights.

8. Viva Questions:

- What basic components of ANN?
- What activation function is used in M-P Model ?

References:

https://pabloinsente.github.io/the-mcculloch-pitts-artificial-neuron-model

https://towardsdatascience.com/mcculloch-pitts-model-5fdf65ac5dd1

Machine Learning
Experiment No. : 5
Naive Bayes classification

Experiment No.5

1. **Aim:** Implement Naive Bayes classification algorithm.

2. **Objectives:**
   Analyse the data, identify the problem and choose relevant algorithm to apply.
   Identify the classification application in machine learning.
3. **Outcomes:**
   Students will be able to understand and implement classification algorithms.

4. **Software Required : R Studio /Python**

5. **Theory:**

Naïve Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems. Naïve Bayes Classifier is one of the simple and most effective Classification algorithms, which helps in building the fast machine learning models that can make quick predictions. It is a probabilistic classifier, which means it predicts based on the probability of an object. Some popular examples of Naïve Bayes Algorithm are spam filtration, Sentimental analysis, and classifying articles.

Bayes' theorem is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability. The formula for Bayes' theorem is given by:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B

P(B|A) is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true

P(A) is Prior Probability: Probability of hypothesis before observing the evidence

P(B) is Marginal Probability: Probability of Evidence.

e.g.

Name	Give Birth	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	yes	mammals
python	no	no	no	no	non-mammals
salmon	no	no	yes	no	non-mammals
whale	yes	no	yes	no	mammals
frog	no	no	sometimes	yes	non-mammals
komodo	no	no	no	yes	non-mammals
bat	yes	yes	no	yes	mammals
pigeon	no	yes	no	yes	non-mammals
cat	yes	no	no	yes	mammals
leopard shark	yes	no	yes	no	non-mammals
turtle	no	no	sometimes	yes	non-mammals
penguin	no	no	sometimes	yes	non-mammals
porcupine	yes	no	no	yes	mammals
eel	no	no	yes	no	non-mammals
salamander	no	no	sometimes	yes	non-mammals
gila monster	no	no	no	yes	non-mammals
platypus	no	no	no	yes	mammals
owl	no	yes	no	yes	non-mammals
dolphin	yes	no	yes	no	mammals
eagle	no	yes	no	yes	non-mammals

A: attributes

M: mammals

N: non-mammals

$$P(A \mid M) = \frac{6}{7} \times \frac{6}{7} \times \frac{2}{7} \times \frac{2}{7} = 0.06$$

$$P(A \mid N) = \frac{1}{13} \times \frac{10}{13} \times \frac{3}{13} \times \frac{4}{13} = 0.0042$$

$$P(A \mid M)P(M) = 0.06 \times \frac{7}{20} = 0.021$$

$$P(A \mid N)P(N) = 0.004 \times \frac{13}{20} = 0.0027$$

Give Birth	Can Fly	Live in Water	Have Legs	Class
yes	no	yes	no	?

P(A|M)P(M) > P(A|N)P(N)

=> Mammals

6. **Output Analysis: (Students should write output analysis based on the output. Specify each output explicitly with output analysis)**

7. **Additional Learning: (Students should write additional learning on their own based on what additionally they learnt after performing the experiment)**

8. **Conclusion:** (Students should write proper conclusion (in the form of discussion)on their own)

Classification is a supervised learning problem. Through this experiment, the need for classification algorithm was recognized and understood, and we implemented a Naive Bayes classification algorithm which uses a probabilistic approach.

9. Viva Questions:

- What is supervised learning?
- What are various classification algorithms?
- What is prior probability of a class?

- **References:**

1. Jake VanderPlas, Python Data Science Handbook: Essential Tools for Working with Data, O'reilly Publication
2. Wes McKinney , Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython, O'reilly Publication
3. Jason Test, PYTHON FOR DATA SCIENCE
4. Jason Test, Python Programming: 3 BOOKS IN 1
5. https://www.datacamp.com/tutorial/naive-bayes-scikit-learn
6. https://www.javatpoint.com/machine-learning-naive-bayes-classifier

Machine Learning

Experiment No.: 6

Implement Support Vector Machine for binary classification

Experiment 6

1. **Aim:** To Implement Support Vector Machine for binary classification.

2. **Objectives:** From this experiment, the student will be able to
   - Familiar with support vector machine.
   - Understand and implement classification algorithms in machine learning.
   - Analyse the large data using SVM.

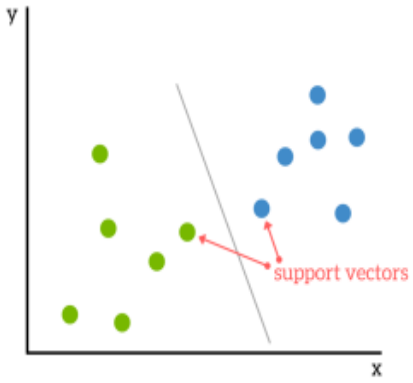
3. **Outcomes:** The learner will be able to

   - Understand and design Support Vector Machines for classification of linear data.

4. **Software Required : Weka/**Java / Python/ Mat lab / R-Tool etc.

5. **Theory:**

A Support Vector Machine (SVM) is a supervised machine learning algorithm that can be employed for both classification and regression purposes. SVMs are more commonly used in classification problems. SVMs are based on the idea of finding a hyper plane that best divides a dataset into two classes, as shown in the image below.



.

Support Vectors

Support vectors are the data points nearest to the hyperplane, the points of a data set that, if removed, would alter the position of the dividing hyperplane. Because of this, they can be considered the critical elements of a data set.

What is a hyperplane?

As a simple example, for a classification task with only two features (like the image above), you can think of a hyperplane as a line that linearly separates and classifies a set of data.

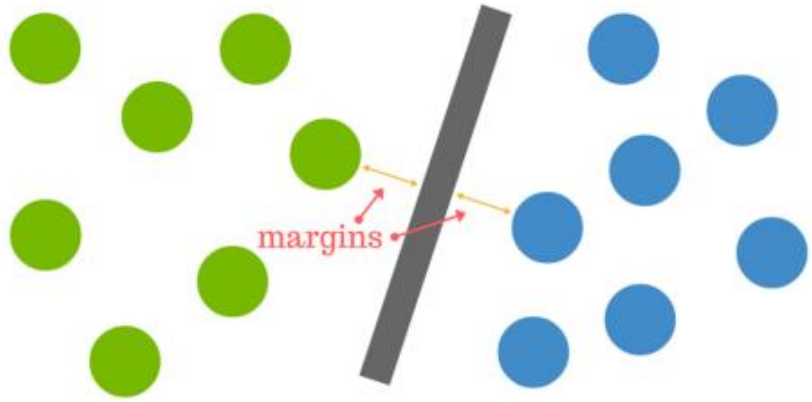
Intuitively, the further from the hyperplane our data points lie, the more confident we are that they have been correctly classified. We therefore want our data points to be as far away from the hyperplane as possible, while still being on the correct side of it.

So when new testing data is added, whatever side of the hyperplane it lands will decide the class that we assign to it.

How do we find the right hyperplane?

Or, in other words, how do we best segregate the two classes within the data?

The distance between the hyperplane and the nearest data point from either set is known as the margin. The goal is to choose a hyperplane with the greatest possible margin between the hyperplane and any point within the training set, giving a greater chance of new data being classified correctly.



Pros & Cons of Support Vector Machines

Pros

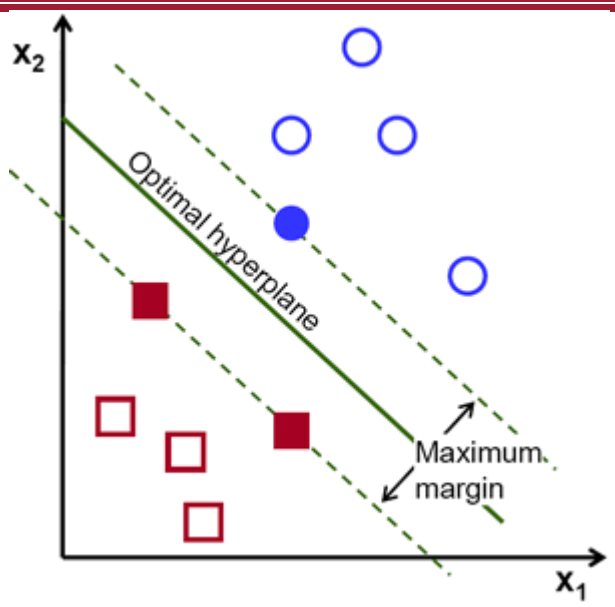
- Accuracy
- Works well on smaller cleaner datasets
- It can be more efficient because it uses a subset of training points

Cons

- Isn't suited to larger datasets as the training time with SVMs can be high
- Less effective on noisier datasets with overlapping classes

6. Procedure/Program:

The operation of the SVM algorithm is based on finding the hyperplane that gives the largest minimum distance to the training examples. Twice, this distance receives the important name of **margin** within SVM's theory. Therefore, the optimal separating hyperplane **maximizes** the margin of the training data.



How is the optimal hyperplane computed?

Step 1: define formally a hyperplane:

$$f(x) = \beta_0 + \beta^T x,$$

where β is known as the **weight vector** and β_0 as the **bias**.

Step 2: The optimal hyperplane can be represented in an infinite number of different ways by scaling of β and β_0. As a matter of convention, among all the possible representations of the hyperplane, the one chosen is

$$|\beta_0 + \beta^T x| = 1$$

where x symbolizes the training examples closest to the hyperplane. In general, the training examples that are closest to the hyperplane are called **support vectors**. This representation is known as the **canonical hyperplane**.

Step 3 : Now, use the result of geometry that gives the distance between a point x and a hyperplane (β, β_0):

$$\text{distance} = \frac{|\beta_0 + \beta^T x|}{\|\beta\|}.$$

Step 5 : In particular, for the canonical hyperplane, the numerator is equal to one and the distance to the support vectors is

$$\text{distance}_{\text{support vectors}} = \frac{|\beta_0 + \beta^T x|}{\|\beta\|} = \frac{1}{\|\beta\|}.$$

Step 6: Recall that the margin introduced in the previous section, here denoted as M, is twice the distance to the closest examples:

$$M = \frac{2}{\|\beta\|}$$

Step 7: Finally, the problem of maximizing M is equivalent to the problem of minimizing a function $L(\beta)$ subject to some constraints. The constraints model the requirement for the hyperplane to classify correctly all the training examples x_i. Formally,

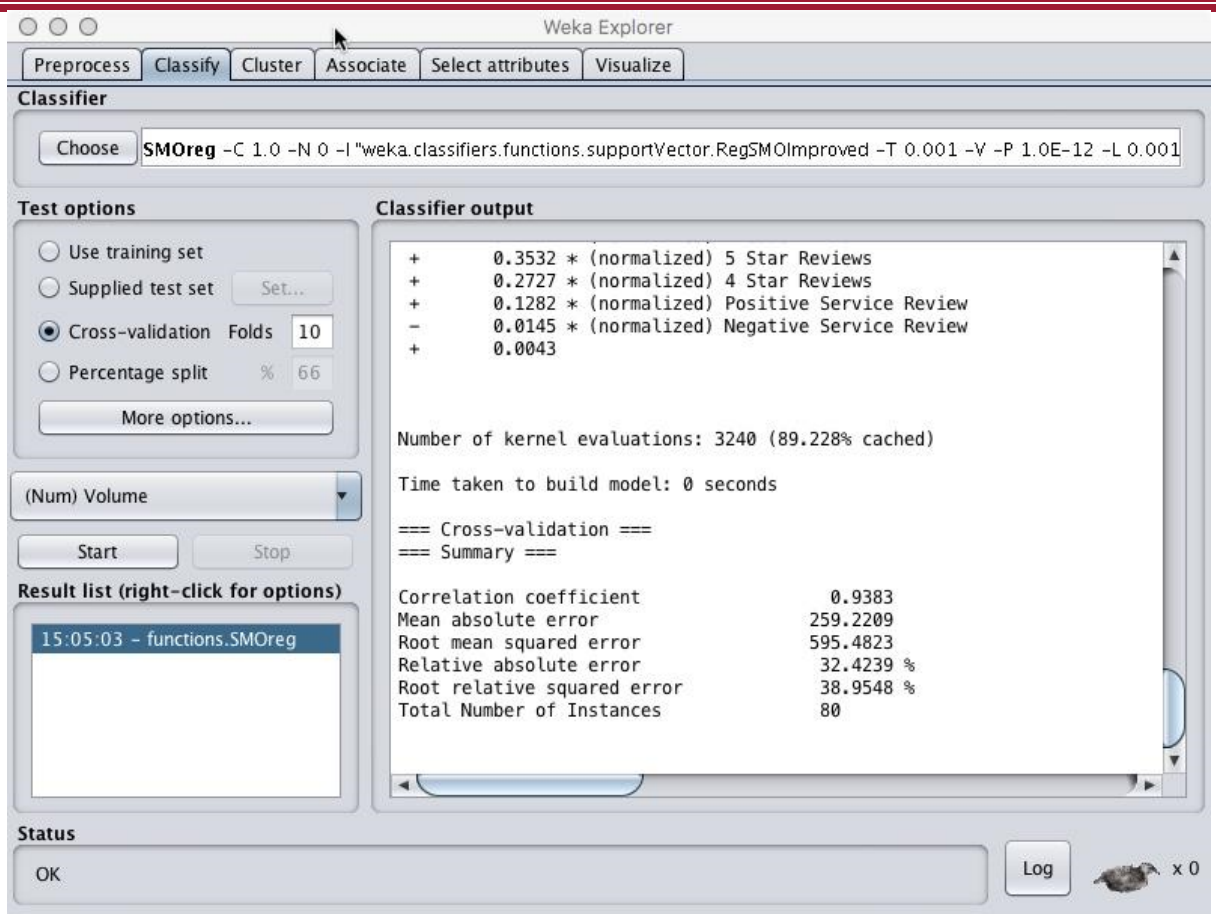
$$\min_{\beta,\beta_0} L(\beta) = \frac{1}{2}\|\beta\|^2 \text{ subject to } y_i(\beta^T x_i + \beta_0) \geq 1 \ \forall i,$$

where y_i represents each of the labels of the training examples.

This is a problem of Lagrangian optimization that can be solved using Lagrange multipliers to obtain the weight vector β and the bias β_0 of the optimal hyperplane.

7. Steps Using Weka:

1. Select Explorer
2. Click: Open File
3. Choose **database.arff** (from DATA Folder)
4. Click: Open
5. Click: Classify
6. Under Classifier: Choose functions>SMOreg
7. Click: Close
8. Confirm cross-validation Folds = 10
9. Right-click on the text box to to the right of the choose button
10. Under Kernel choose Polykernel
11. Click Close
12. Set c = 1
13. Set Filter Type to Normalize training data
14. Click OK
15. Click Start



8. Results:

The operation of the SVM algorithm is based on finding the hyperplane that gives the largest minimum distance to the training examples. Twice, this distance receives the important name of **margin** within SVM's theory. Therefore, the optimal separating hyperplane **maximizes** the margin of the training data.

9. Conclusion:

The different classification algorithms of data mining were studied and one among them named support vector machine algorithm was implemented. The need for classification algorithm was recognized and understood.

10. Viva Questions:

- What is hyperplane?
- What are the different kernel functions?
- What is the role of maximum margin in SVM?

11. References:

1. Tom M.Mitchell “Machine Learning” McGraw Hill
2. *Neural Networks and Deep Learning* by Michael Nielsen: A great resource for understanding the theoretical aspects of neural networks.

Machine Learning
Experiment No.: 7
Implement K-means clustering algorithm.

Experiment 7

1. **Aim:** Implement K-means clustering algorithm.

2. **Objectives:** From this experiment, the student will be able to
   - Analyse the data, identify the problem and choose relevant algorithm to apply
   - Understand and implement classical clustering algorithms in data mining
   - Identify the application of clustering algorithm in data mining
3. **Outcomes:** The learner will be able to
   - Assess the strength and weaknesses of algorithms
   - Identify, formulate and solve engineering problems
   - Analyse the local and global impact of data mining on individuals, organizations and society
4. **Software Required:** Python/R/Java etc.
5. **Theory:**

Clustering is dividing data points into homogeneous classes or clusters:

- Points in the same group are as similar as possible
- Points in different group are as dissimilar as possible

When a collection of objects is given, we put objects into group based on similarity.
Clustering Algorithms:
A Clustering Algorithm tries to analyse natural groups of data on the basis of some similarity. It locates the centroid of the group of data points. To carry out effective clustering, the algorithm evaluates the distance between each point from the centroid of the cluster. The goal of clustering is to determine the intrinsic grouping in a set of unlabelled dataTheory:

K-means Clustering

K-means (Macqueen, 1967) is one of the simplest unsupervised learning algorithms that solve the well-known clustering problem. K-means clustering is a method of vector quantization, originally from signal processing, that is popular for cluster analysis in data mining.

6. Procedure:

Input:

K: the number of clusters D: a data set containing n objects.

Output: A set of k clusters.

I. Arbitrarily choose K objects from D as the initial cluster centers
II. Partition of objects into k non-empty subsets
III. Identifying the cluster centroids (mean point) of the current partition.
IV. Assigning each point to a specific cluster
V. Compute the distances from each point and allot points to the cluster where the distance from the centroid is minimum.
VI. After re-allotting the points, find the centroid of the new cluster formed.

7. Conclusion:

The different clustering algorithms of data mining were studied and one among them named k-means clustering algorithm was implemented using …….Language. The need for clustering algorithm was recognized and understood.

8. Additional Learning:

9. Viva Questions:

1. What are different clustering techniques?
2. What is difference between K-means and K-medoids?
3. What is dendogram?

10. References:

- Han, Kamber, "Data Mining Concepts and Techniques", Morgan Kaufmann 3rd Edition
- M.H. Dunham, "Data Mining Introductory and Advanced Topics", Pearson Education

Machine Learning
Experiment No.: 8
Agglomerative Clustering

Experiment -8

1. **Aim:** Implementing Agglomerative Clustering.
2. **Objectives:**
   - Understand the Agglomerative Clustering.
   - Explore various machine learning tools and techniques .
3. **Outcomes:**
   Students will be able to use Agglomerative Clustering techniques for classification.

3. **Software (tools/packages) Required** : Python

4. **Theory:**

Agglomerative Clustering is a type of hierarchical clustering algorithm. It is an unsupervised machine learning technique that divides the population into several clusters such that data points in the same cluster are more similar and data points in different clusters are dissimilar.

1. Points in the same cluster are closer to each other.
2. Points in the different clusters are far apart.

Agglomerative Clustering is a bottom-up approach, initially, each data point is a cluster of its own, further pairs of clusters are merged as one moves up the hierarchy.

Steps of Agglomerative Clustering:

a) Initially, all the data-points are a cluster of its own.
b) Take two nearest clusters and join them to form one single cluster.
c) Proceed recursively step 2 until you obtain the desired number of clusters.

obtain the desired number of clusters, the number of clusters needs to be reduced from initially being n cluster (n equals the total number of data-points). Two clusters are combined by computing the similarity between them.

There are some methods which are used to calculate the similarity between two clusters:

a) Distance between two closest points in two clusters.
b) Distance between two farthest points in two clusters.
c) The average distance between all points in the two clusters.
d) Distance between centroids of two clusters.
e) There are several pros and cons of choosing any of the above similarity metrics.

5. Output Analysis: (Students should write output analysis based on the output. Specify each output explicitly with output analysis)

6. Additional Learning: (Students should write additional learning on their own based on what additionally they learnt after performing the experiment)

7. Conclusion

8. Viva Questions:

- What is mean Agglomerative Clustering.?
- What Computing Distance Matrix are used in Agglomerative Clustering?

References:

1. https://towardsdatascience.com/agglomerative-clustering-and-dendrograms-explained-29fc12b85f23.
2. https://www.javatpoint.com/hierarchical-clustering-in-machine-learning

Machine Learning Lab
Experiment No.: 9
Case study of Reinforcement learning
Elevator Dispatching

Experiment -9

Aim: Case study of Reinforcement learning Elevator Dispatching

Theory:

Waiting for an elevator is a situation with which we are all familiar. We press a button and then wait for an elevator to arrive traveling in the right direction. We may have to wait a long time if there are too many passengers or not enough elevators. Just how long we wait depends on the dispatching strategy the elevators use to decide where to go. For example, if passengers on several floors have requested pickups, which should be served first? If there are no pickup requests, how should the elevators distribute themselves to await the next request? Elevator dispatching is a good example of a stochastic optimal control problem of economic importance that is too large to solve by classical techniques such as dynamic programming.

Crites and Barto (1996; Crites, 1996) studied the application of reinforcement learning techniques to the four-elevator, ten-floor system shown in Figure 11.8. Along the right-hand side are pickup requests and an indication of how long each has been waiting. Each elevator has a position, direction, and speed, plus a set of buttons to indicate where passengers want to get off. Roughly quantizing the continuous variables, Crites and Barto estimated that the system has over 10^{22} states. This large state set rules out classical dynamic programming methods such as value iteration. Even if one state could be backed up every microsecond it would still require over 1000 years to complete just one sweep through the state space.

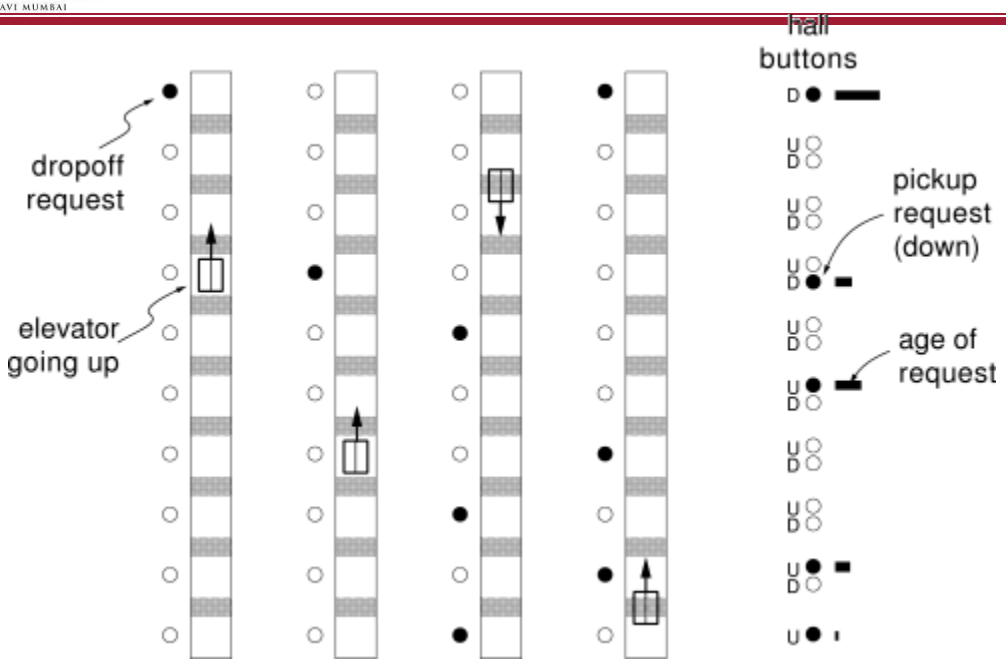


Figure 11.8: Four elevators in a ten-story building.

In practice, modern elevator dispatchers are designed heuristically and evaluated on simulated buildings. The simulators are quite sophisticated and detailed. The physics of each elevator car is modeled in continuous time with continuous state variables. Passenger arrivals are modeled as discrete, stochastic events, with arrival rates varying frequently over the course of a simulated day. Not surprisingly, the times of greatest traffic and greatest challenge to the dispatching algorithm are the morning and evening rush hours. Dispatchers are generally designed primarily for these difficult periods.

The performance of elevator dispatchers is measured in several different ways, all with respect to an average passenger entering the system. The average *waiting time* is how long the passenger waits before getting on an elevator, and the average *system time* is how long the passenger waits before being dropped off at the destination floor. Another frequently encountered statistic is the percentage of passengers whose waiting time exceeds 60 seconds. The objective that Crites and Barto focused on is the average *squared waiting time*. This objective is commonly used because it tends to keep the waiting times low while also encouraging fairness in serving all the passengers.

Crites and Barto applied a version of one-step Q-learning augmented in several ways to take advantage of special features of the problem. The most important of these concerned the formulation of the actions. First, each elevator made its own decisions independently of the others. Second, a number of constraints were placed on the decisions. An elevator carrying passengers could not pass by a floor if any of its

passengers wanted to get off there, nor could it reverse direction until all of its passengers wanting to go in its current direction had reached their floors. In addition, a car was not allowed to stop at a floor unless someone wanted to get on or off there, and it could not stop to pick up passengers at a floor if another elevator was already stopped there. Finally, given a choice between moving up or down, the elevator was constrained always to move up (otherwise evening rush hour traffic would tend to push all the elevators down to the lobby). These last three constraints were explicitly included to provide some prior knowledge and make the problem easier. The net result of all these constraints was that each elevator had to make few and simple decisions. The only decision that had to be made was whether to stop at a floor that was being approached and that had passengers waiting to be picked up. At all other times, no choices needed to be made.

That each elevator made choices only infrequently permitted a second simplification of the problem. As far as the learning agent was concerned, the system made discrete jumps from one time at which it had to make a decision to the next. When a continuous-time decision problem is treated as a discrete-time system in this way it is known as a *semi-Markov* decision process. To a large extent, such processes can be treated just like any other Markov decision process by taking the reward on each discrete transition as the integral of the reward over the corresponding continuous-time interval. The notion of return generalizes naturally from a discounted sum of future rewards to a discounted *integral* of future rewards:

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \qquad \text{becomes} \qquad R_t = \int_0^{\infty} e^{-\beta\tau} r_{t+\tau}\, d\tau$$

where r_t on the left is the usual immediate reward in discrete time and $r_{t+\tau}$ on the right is the instantaneous reward at continuous time $t+\tau$. In the elevator problem the continuous-time reward is the negative of the sum of the squared waiting times of all waiting passengers. The parameter $\beta > 0$ plays a role similar to that of the discount-rate parameter $\gamma \in [0, 1)$.

The basic idea of the extension of Q-learning to semi-Markov decision problems can now be explained. Suppose the system is in state s and takes action a at time t_1, and then the next decision is required at time t_2 in state s'. After this discrete-event transition, the semi-Markov Q-learning backup for a tabular action-value function, Q, would be:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[\int_{t_1}^{t_2} e^{-\beta(\tau - t_1)} r_\tau \, d\tau + e^{-\beta(t_2 - t_1)} \min_{a'} Q(s', a') \right]$$

Note how $e^{-\beta(t_2-t_1)}$ acts as a variable discount factor that depends on the amount of time between events. This method is due to Bradtke and Duff (1995).

One complication is that the reward as defined--the negative sum of the squared waiting times--is not something that would normally be known while an actual elevator was running. This is because in a real elevator system one does not know how many people are waiting at a floor, only how long it has been since the button requesting a pickup on that floor was pressed. Of course this information is known in a simulator, and Crites and Barto used it to obtain their best results. They also experimented with another technique that used only information that would be known in an on-line learning situation with a real set of elevators. In this case one can use how long since each button has been pushed together with an estimate of the arrival rate to compute an *expected* summed squared waiting time for each floor. Using this in the reward measure proved nearly as effective as using the actual summed squared waiting time.

For function approximation, a nonlinear neural network trained by backpropagation was used to represent the action-value function. Crites and Barto experimented with a wide variety of ways of representing states to the network. After much exploration, their best results were obtained using networks with 47 input units, 20 hidden units, and two output units, one for each action. The way the state was encoded by the input units was found to be critical to the effectiveness of the learning. The 47 input units were as follows:

- 18 units: Two units encoded information about each of the nine hall buttons for down pickup requests. A real-valued unit encoded the elapsed time if the button had been pushed, and a binary unit was on if the button had not been pushed.

- 16 units: A unit for each possible location and direction for the car whose decision was required. Exactly one of these units was on at any given time.

- 10 units: The location of the other elevators superimposed over the 10 floors. Each elevator had a "footprint" that depended on its direction and speed. For example, a stopped elevator caused activation only on the unit corresponding to its current floor, but a moving elevator caused activation on several units corresponding to

the floors it was approaching, with the highest activations on the closest floors. No information was provided about which one of the other cars was at a particular location.

- 1 unit: This unit was on if the elevator whose decision was required was at the highest floor with a passenger waiting.

- 1 unit: This unit was on if the elevator whose decision was required was at the floor with the passenger who had been waiting for the longest amount of time.

- 1 unit: Bias unit was always on.

Two architectures were used. In RL1, each elevator was given its own action-value function and its own neural network. In RL2, there was only one network and one action-value function, with the experiences of all four elevators contributing to learning in the one network. In both cases, each elevator made its decisions independently of the other elevators, but shared a single reward signal with them. This introduced additional stochasticity as far as each elevator was concerned because its reward depended in part on the actions of the other elevators, which it could not control. In the architecture in which each elevator had its own action-value function, it was possible for different elevators to learn different specialized strategies (although in fact they tended to learn the same strategy). On the other hand, the architecture with a common action-value function could learn faster because it learned simultaneously from the experiences of all elevators. Training time was an issue here, even though the system was trained in simulation. The reinforcement learning methods were trained for about four days of computer time on a 100 mips processor (corresponding to about 60,000 hours of simulated time). While this is a considerable amount of computation, it is negligible compared with what would be required by any conventional dynamic programming algorithm.

The networks were trained by simulating a great many evening rush hours while making dispatching decisions using the developing, learned action-value functions. Crites and Barto used the Gibbs softmax procedure to select actions as described in Section 2.3, reducing the "temperature" gradually over training. A temperature of zero was used during test runs on which the performance of the learned dispatchers was assessed.

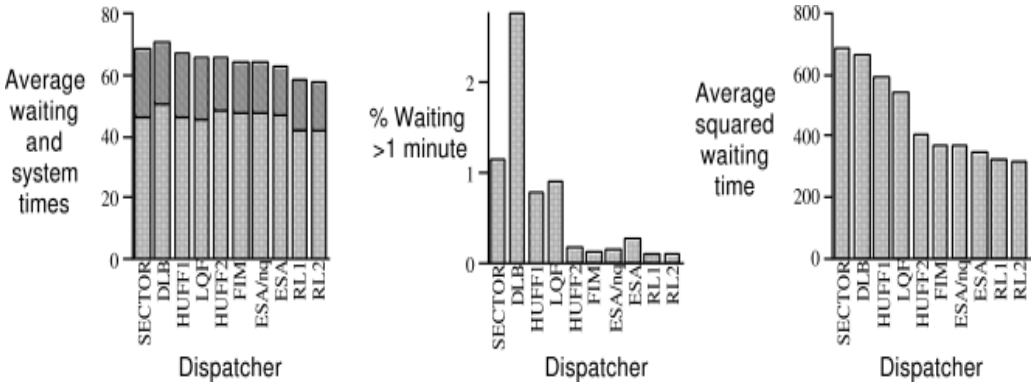


Figure 2: *Comparison of elevator dispatchers. The SECTOR dispatcher is similar to what is used in many actual elevator systems. The RL1 and RL2 dispatchers were constructed through reinforcement learning*.

Figure 2 shows the performance of several dispatchers during a simulated evening rush hour, what researchers call *down-peak* traffic. The dispatchers include methods similar to those commonly used in the industry, a variety of heuristic methods, sophisticated research algorithms that repeatedly run complex optimization algorithms on-line (Bao et al., 1994), and dispatchers learned by using the two reinforcement learning architectures. By all of the performance measures, the reinforcement learning dispatchers compare favorably with the others. Although the optimal policy for this problem is unknown, and the state of the art is difficult to pin down because details of commercial dispatching strategies are proprietary, these learned dispatchers appeared to perform very well.

- **References:**

1. http://incompleteideas.net/book/ebook/node107.html

Machine Learning

Experiment No.: 10

VLAB- RANDOM FOREST/ BACK PROPAGATION

Experiment 10

1. Aim: Implementing Agglomerative Clustering.

2. Objectives:

- Understand the Agglomerative Clustering.
- Explore various machine learning tools and techniques .

3. **Outcomes:**

Students will be able to use Agglomerative Clustering techniques for classification.

4. **Software (tools/packages) Required**: Students should write from the VLAB.

5. **Theory:** Students should write from the VLAB.

6. Results:

7. Conclusion: Students should write the conclusion in their own words

- **References:**

https://vlab.spit.ac.in/ai/#/experiments

Machine Learning
Experiment No.: 11

CBS-Optical Character Recognition